Frontiers in Computing: Discussion 9

Rento Saijo

2026-10-27

Why version control matters

- A document, design, or program rarely reaches its final form in one sitting. We create files, revise them, and sometimes discover that an earlier choice worked better. Version control records selected states of a project together with information about who made a change, when it was recorded, and why. We can compare versions to see precisely what changed instead of relying on filenames such as "final-v2." The first Git Basics video stresses that this history is useful beyond programming: a designer changing a logo and a writer correcting a document face the same need to understand earlier work.
- Collaboration makes that record more valuable. Two people may edit adjacent files or even the same lines before either sees the other's work. A version control system retains their separate contributions and helps combine them through a *merge*; a collision still needs a human decision about the desired result. Git is one implementation of version control. Unlike a system that needs a central server before we can save history, Git can create commits in a repository on our own computer. A shared server becomes useful when we want colleagues to exchange those commits, but local history does not depend on an internet connection.
- The second Git Basics video introduces a project as a sequence of snapshots that can also branch. Each collaborator can start from a shared commit, work independently, and later integrate the results. This explains why the project history is more than a stack of backup copies: commits identify states and their parent commits, allowing Git to relate simultaneous lines of work. Git's small commands also let us choose exactly which changes belong in a commit. A graphical app can present the same history visually, while the underlying model remains the same.
- The local design also changes what "the project" means in a team. Dana and Doug need not take turns at one keyboard or wait for a server to accept each step: each can keep a repository with the project's history and make local commits. Sharing later means exchanging commits and reconciling any branches that have diverged. The copies give collaborators independence, but they also make coordination necessary. A teammate cannot review a new local commit until it is shared, and identical filenames do not prove that two repositories have the same committed state.

Recording work in a local repository

- The third Git Basics video begins with two pieces of attribution: `git config --global user.name` and `git config --global user.email`. Git places these values on commits, so an email in a public repository may become public. They are not credentials for a hosting account; authentication to GitHub is a separate step. We can start a repository with `git init`, either in the current project directory or with a new directory name. Git stores its history and control information in the hidden `.git` directory, while the ordinary files we edit form the *working tree*. Deleting `.git` removes that local repository's history even if the visible project files remain.

- The basic cycle is edit, inspect, stage, and commit. `git status` shows the current branch and distinguishes untracked files, changes waiting in the working tree, and changes staged for the next commit. We use `git add file` to place a chosen file's current contents in the *staging area*, also called the index. `git commit -m "message"` then records a snapshot of the staged project state with an explanatory message. A commit is local until it is sent elsewhere. Repeating this cycle makes a useful history of completed steps, rather than saving every keystroke automatically.
- Coder Coder makes that cycle concrete with a small website. Just after `git init`, `git status` reports no commits and no changes. Creating `index.html` and `style.css` makes them *untracked*; Git sees the paths but has no committed version to compare with. After `git add index.html style.css`, status lists them under changes to be committed. The first `git commit` becomes the root of this local history, and status returns to a clean working tree. That clean report means Git detects no pending changes, not that the site has been tested or that unrelated ignored files are safe elsewhere.
- Staging has a timing consequence that Coder Coder demonstrates with an HTML file. Suppose we add an initial page, run `git add index.html`, and then write another heading. The index still contains the page as it looked at the earlier `add`; the new heading remains unstaged. Running `git add index.html` again updates the staged contents. The same file can therefore appear in both the staged and unstaged parts of `git status`. This is why staging is useful for constructing a coherent commit, but it also means that "I already added the file" does not guarantee that our latest edits will be included.
- Before committing, `git diff` shows tracked changes between the working tree and the staging area; `git diff --staged` compares the staged contents with the last commit. Together with `git status`, these checks tell us what we have changed and what the next commit will actually contain. Afterward, `git log` displays the history with authors, dates, messages, and commit identifiers, while `git log --oneline` gives a compact view. Each ordinary commit points to its parent, so we can follow the ancestry of a branch. A concise message should describe the change well enough for a future reader to understand why that point in the history exists.
- The index also lets us decide the scope of a commit. `git add .` stages eligible changes under the current directory, which is convenient but can include an unintended file or deletion. Naming paths is more selective, and Git can even stage chosen portions of a text file when two ideas are interleaved. The tutorial also uses `git commit -am "message"` to stage and commit tracked modifications in one command. That shortcut does not include newly untracked files, and it reduces our chance to review the exact staged set. The deliberate inspection steps make a commit easier to explain and easier to reverse later.
- Coder Coder's file move illustrates another useful distinction: Git records files and paths, not empty folders as independent objects. Moving a style sheet into a new directory may first appear as a deletion and an untracked path; after staging, Git may recognize the change as a rename by comparing file contents. For generated build files or a local file containing secrets, we can place patterns in `.gitignore` so untracked matches stay out of normal staging. The ignore file itself can be committed for teammates. An already tracked file remains tracked despite a new ignore rule; we must remove it from the index separately if we want future commits to stop tracking it, and a past public commit is not erased by that change.
- In the tutorial, a local environment file and a generated build directory disappear from normal status after matching ignore patterns are added. That disappearance is a useful check, but it should not be mistaken for encryption or access control. The files still exist on the computer, and a forced add could include an ignored path. If a sensitive file was already committed, adding it to `.gitignore` cannot take back copies others may have downloaded. We would stop tracking the path for future commits, then treat any exposed credential as compromised and replace it. The point of the ignore rule is to prevent an accidental future addition, not to repair a published secret.

Branches, recovery, and shared repositories

- A branch is a movable reference to a commit, not a complete second copy of the project. In the tutorial, `main` points to the current shared line of work, while a `welcome` branch lets us add a greeting without advancing `main`. We can create and switch in one step with `git switch -c welcome`; `git branch` lists branches and `git switch main` returns to the main line. Switching normally updates the working tree to match the selected branch's commit, subject to safeguards for local edits. New commits move the current branch's reference forward. This makes a feature branch a manageable place to finish one change before integrating it.
- The log makes those movements visible. Before the greeting is committed, `main` and `welcome` can point to the same commit. After a commit on `welcome`, only that branch moves; `HEAD` normally identifies the branch we currently have checked out. When we switch back to `main`, the newly added paragraph disappears from the working file because the selected branch still refers to the earlier state. Switching back to `welcome` restores the paragraph from its commit. Nothing has been copied back and forth by hand: Git is using the branch references and recorded snapshots to present the selected version.
- To integrate the finished greeting, we switch to the destination branch and run `git merge welcome`. If `main` has not advanced since `welcome` branched, Git can *fast-forward*: it simply moves the `main` reference to the newer commit. When both lines have advanced, Git combines their changes, usually recording a merge commit; incompatible edits require us to resolve a conflict. The result should be reviewed because a successful automatic merge does not prove the combined program behaves correctly. Once its work is incorporated, the feature branch can be deleted without deleting the commits now reachable from `main`.
- A conflict is easiest to picture when two collaborators independently rewrite the same welcome sentence. Git can identify the competing versions and the common starting point, yet it cannot know whether the site should use one wording, the other, or a revised sentence that combines both ideas. We edit the conflicted file to the desired result, stage that resolution, and finish the merge. If the collaborators changed separate files, the merge is more likely to complete automatically. In either case, the recorded history explains how the resulting state was reached, while review and testing address whether that state is actually good.
- The repair tool depends on whether a mistake has entered history. If we accidentally delete a tracked style sheet but have not staged or committed that deletion, `git restore path/to/style.css` restores its indexed version to the working tree. It also discards other unstaged edits to that path, so we should inspect `git diff` first. If a deletion was committed and shared, `git revert <commit>` creates a new commit that reverses its effect while preserving the published history. This is safer for collaborators than rewriting a commit they may already have fetched. Untracked files have no committed version for ordinary `git restore` to recover, and GitHub cannot restore edits that were never committed and pushed.
- There is an intermediate case between editing and committing: a change may already be staged but still be unwanted in the next snapshot. `git restore --staged path/to/file` removes it from the index while leaving the working file as edited. We can then revise it, stage it later, or discard its working-tree edit separately. By contrast, a committed deletion appears in `git log`; reverting that commit records the restoration as new history. The distinction matters because "undo" is not one operation in Git. We first identify whether the change is in the working tree, the index, a local commit, or a published commit.
- GitHub adds a hosted *remote* repository that others can access, review, and clone. A local repository is fully useful on its own; connecting it to a remote gives us a place to exchange commits and a second copy of the history we actually publish. We can create an empty GitHub repository, run `git remote add origin <url>`, and send our current `main` branch with `git push -u origin main`. Here `origin` is a conventional remote name, and `-u` records an upstream relationship for later pushes. If the new GitHub repository already contains a README commit, its history is not the same empty starting point; we should integrate that remote history before pushing rather than assume a simple first push will work.

- Pushing transfers commits, not every file that happens to be open in an editor. An unstaged edit, a staged edit awaiting commit, and even a local commit that has not been pushed are absent from GitHub's copy. Conversely, `git fetch` obtains remote updates without integrating them into the current branch, while `git pull` fetches and then integrates according to the repository's chosen strategy. A teammate can *clone* the hosted repository to get a local copy and contribute through the same edit–stage–commit cycle. GitHub makes distribution and collaboration convenient, but it does not replace checking what we commit, deciding what should be public, or resolving competing changes.
- A GitHub repository can be public or private, and that choice governs who can see the hosted copy. It does not change the logic of a local commit: neither visibility setting uploads a working-tree edit by itself. A remote with its own README already has an initial commit, so a separately initialized local project may need to bring in that remote history before the first ordinary push succeeds. The alternative shown in the Desktop lesson is to create the repository on GitHub first and *clone* it. In that order, the local copy begins with the remote's existing history and can build on it directly.

The same workflow in GitHub Desktop

- Coder Coder's Desktop tutorial starts by creating a GitHub repository and choosing its visibility. *Public* makes the repository visible to everyone; *private* restricts access to people we authorize. An optional README can introduce the project, a `.gitignore` can exclude appropriate untracked files, and a license states reuse terms. Adding a README while creating the repository also creates its first commit. Cloning that repository in GitHub Desktop then brings its committed files and history to a local folder, where an editor can change them. The labels and buttons differ from a terminal, yet the remote and local repositories remain distinct.
- Desktop's Changes view corresponds to `git status` and the diff views: it shows modified and new files, with added and removed lines. We can select which files, and in the demonstrated case which lines, belong in the next commit. The commit message gives the chosen snapshot a useful explanation. After committing, the History view shows the new local commit, but GitHub's website still shows the old history until we *push origin*. This separation lets us make several local commits before sharing them. The inverse direction matters too: fetching checks and downloads remote commits, while pulling incorporates them into our current local branch.
- In the demonstration, a README edit and a newly created text file appear together in Changes. The presenter excludes the text file and one line of the README from the first commit, giving that commit a focused message. The omitted work stays local for a later decision. After the commit, History shows the new entry but the GitHub website still shows only its earlier initial commit; pushing origin closes that gap. This is the same separation we saw with `git add`, `git commit`, and `git push`, expressed through selected checkboxes and a history view rather than typed commands.
- Desktop also makes the three recovery situations visible. *Discard changes* removes an uncommitted working-tree edit; *Undo* can return the newest unpushed commit's changes to the Changes view; and *Revert changes in commit* adds a new inverse commit after a change has been published. We then push that revert so the remote receives the correction. In each interface, the question is the same: has this change merely been edited, committed locally, or shared with others? Choosing the answer first prevents a routine correction from silently losing work or disrupting a collaborator's history.
- For example, if the test text file was committed locally by mistake but not pushed, Desktop's Undo returns it to the uncommitted Changes view so we can decide how to proceed. Once that commit has been pushed, another collaborator may already have it. Revert then records a new commit that removes the file while retaining the original addition in the shared history. The new commit itself must be pushed to appear on GitHub. Thus the graphical controls do not eliminate Git's history rules; they surface the appropriate operation at each stage of sharing.